

LAPORAN PRAKTIKUM

Pemrograman Berorientasi Objek

Relasi Antar Objek (Class Diagram & Asosiasi)

disusun Oleh:

Nama: RIFKI MAULANA

NIM: 2511533007

Kelas: A

DOSEN PENGAMPU: NURFIAH, S.ST, M.Kom

ASISTEN PRAKTIKUM: M. DAWI SYAUQI



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 30 September 2026

A. Langkah Kerja Praktikum

1. Langkah 1 Mengunci entitas transaksi

Pada file transaksi.java ubah variable menjadi private dan berikan getter untuk data yang perlu dibaca. Objek transaksi bersifat immutable (tidak boleh diubah setelah dibuat. sehingga tidak perlu setter

```
package Pekan3;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // constructor
    public Transaksi(String id, String jenis, double nominal){
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() { return idTransaksi;}
    public String getJenis() { return jenis;}
    public double getNominal() { return nominal;}

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | jenis: " + jenis + " | Nominal: Rp" + nominal);
    }
}
```

- Buat var idTransaksi, jenis dan nominal jadi private
- Berikan getter untuk membaca var

2. Langkah 2: Mengunci entitas rekening dan menambahkan PIN

Pada file rekening.java amankan saldo dan tambahkan atribut PIN

```

package Rekening;
import java.util.ArrayList;

public class Rekening {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif

    // Implementasi Asosiasi (1-to-many)
    private ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, String pinAwal, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
        // Validasi PIN di dalam Constructor
        if (pinAwal.length() != 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }
        // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    // 4. Method otentikasi internal (Validasi Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
}

```

- Buat nomorRekening, namaPemilik, saldo, riwayatTransaksi menjadi private dan tambahkan private pin
- Modif constructor untuk menerima pin dengan catatan bahwa pin harus berjumlah 6 karakter dan jika tidak menginputkan 6 karakter maka akan menggunakan pin default
- Buat getter untuk atribut yang boleh dibaca oleh public yaitu untuk nomorRekening dan namaPemilik
- Buat method untuk validasi pin(validasi enkapsulasi)
 Mengapa tidak membuat setSaldo?
 = karena nantinya pengguna bisa mengubah saldo, saldo tidak boleh diubah oleh pengguna
 Mengapa tidak membuat getPin?
 = tujuan dari pembuatan pin adalah untuk menjaga akun pengguna dari orang lain, jika dibuat getPin maka pin bisa dilihat

B. Tugas/Latihan praktikum

Tugas : Mengamankan CLI dengan Otentikasi

1. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user memasukkan PIN (String angka 6 digit).

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan PIN(6 digit angka): ");
    String pin = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();

    if (saldo < 50000) {
        System.out.println("saldo kurang");
    } else {
        akunAktif = new Rekening(no, nama, saldo, pin);
        daftarRekening.add(akunAktif);
    }
    break;
```

- Tambahkan inputan pin dan tambahkan pin pada parameter akunAktif
2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.
 - Panggil fungsi akunAktif.otentikasi(pinYangDiinput).
 - Jika mengembalikan nilai true, jalankan transaksi.
 - Jika false, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

```

case 3:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        System.out.println("Masukkan PIN: ");
        String inputPin = input.nextLine();
        if (akunAktif.otentikasi(inputPin) == true) {
            System.out.println("Masukkan nominal tarik tunai: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;

```

```

case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        System.out.println("Masukkan PIN: ");
        String inputPin = input.nextLine();
        if (akunAktif.otentikasi(inputPin) == true) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;

```

- Program melakukan if untuk melihat apakah method otentikasi dengan argument inputPin bernilai true
- Jika tidak maka tidak akan menjalankan method yang diharapkan(tarikTunai dan cetakMutasi)